Session 25 Homework — Solution Walkthrough

Status: PREPARED DRAFT. Published with the official solution, for independent study.

What the homework asked you to prove

A real Redux Toolkit current-user snapshot (this cohort's genuine first real use of Redux), wired into the protected layout so a Client Component can show the signed-in user and log them out; resolved real names (instead of raw ids) for Issue Detail's reporter/assignee; then, as a real STRETCH transfer of the generic Server-Action-form pattern already taught, a real Create Workspace flow and a full real Workspace Settings page (search, add, change role, remove a member) — reachable from both desktop and the real mobile drawer. The CHALLENGE asked for a real, independent written synthesis, not new code.

What the official solution changes

CORE: `web/src/store/` (`currentUserSlice.ts`, `store.ts`, `StoreProvider.tsx`) — the one real justified Redux use case this app needs; the protected `(app)/layout.tsx` now fetches `GET /auth/me` and hydrates the store; `AccountBar.tsx` reads the current user from Redux and exposes `logoutAction`; a real `/account` page; Issue Detail now resolves real names via `GET /users/:id`.

STRETCH: `lib/actions/workspaces.ts` (`createWorkspaceAction`), `workspaces/new/` (the real Create Workspace form and page), a link from the Workspaces list; `lib/actions/workspace- members.ts` (`searchUsersAction`, `addMemberAction`, `updateMemberRoleAction`, `removeMemberAction`), `workspaces/[id]/settings/` (`AddMemberForm.tsx`, `MemberRow.tsx`, `page.tsx`), a Settings link from Workspace Detail (desktop) and the mobile `WorkspaceSidebar` drawer.

CHALLENGE: no code — a real, independent written answer.

Files changed

- `web/src/store/currentUserSlice.ts`, `store.ts`, `StoreProvider.tsx` (new)
- `web/src/app/(app)/layout.tsx` (CORE: fetch + hydrate + render `AccountBar`)
- `web/src/components/layout/AccountBar.tsx` (new)
- `web/src/app/(app)/account/page.tsx` (new)
- `web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx` (CORE: `resolveName` helper + `Promise.all`)
- `web/src/lib/actions/workspaces.ts` (new)
- `web/src/app/(app)/workspaces/new/CreateWorkspaceForm.tsx`, `page.tsx` (new)
- `web/src/app/(app)/workspaces/page.tsx` (STRETCH: "+ New workspace" link)
- `web/src/lib/actions/workspace-members.ts` (new)
- `web/src/app/(app)/workspaces/[id]/settings/AddMemberForm.tsx`, `MemberRow.tsx`, `page.tsx` (new)
- `web/src/app/(app)/workspaces/[id]/page.tsx` (STRETCH: Settings link)
- `web/src/components/layout/WorkspaceSidebar.tsx` (STRETCH: mobile Settings link)

Step-by-step implementation

1. **`currentUserSlice`:** one reducer, `setCurrentUser` — the current user is always set whole, never field-by-field, since it is set once at hydration and cleared on logout.
2. **`store.ts`:** `makeStore()` is a real FACTORY, called once per real request inside `StoreProvider` — a module-level singleton would leak one real user's state into another's request on a real server handling many requests concurrently.
3. **`StoreProvider`:** a real `"use client"` bridge — the Server Component root layout fetches the real current user (it can `await`); this component creates the real store and dispatches the value in, once, via `useRef` so it never happens twice.
4. **Layout wiring:** the protected layout calls `apiFetch("/auth/me", { token })` — the real first live exercise of the token support and 401-handling LIVE already built — and passes the result into `StoreProvider`.
5. **`logoutAction` / `AccountBar`:** `logoutAction` takes no arguments at all (unlike `loginAction` / `registerAction`) since a logout `<form>` carries no real fields; `AccountBar` reads `state.currentUser.user` specifically, not the whole slice.
6. **`/account` page:** an independent Server Component, calling `requireSessionToken()` + `apiFetch("/auth/me", { token })` itself — the identical real "each page fetches what it needs" convention this app already uses everywhere.
7. **Resolving names:** `resolveName(id, token)` calls `GET /users/:id`, falling back to `` `user #${id}` `` (never a crash) on a non-ok response; reporter and assignee are resolved together via Promise.all`, since they are independent real lookups.
8. **`createWorkspaceAction`:** validates `name` / `slug` are present, then calls `POST /workspaces`; a real duplicate-slug `409` becomes an inline form error, never a thrown exception — the identical real mistake `registerAction`'s duplicate-email case already warned against.
9. **`searchUsersAction`:** called DIRECTLY from `AddMemberForm`, as a plain async function, on every real keystroke — a genuinely different way to use a Server Action from the `useActionState` / `<form action>` pattern taught so far.
10. **`addMemberAction` /member-management actions:** the identical real `requireSessionToken()` + `apiFetch(..., { token })` + `revalidatePath` pattern, reused for add/change-role/remove; the latter two take plain `workspaceId` / `userId` arguments (never a `<form>` submission) since each row already has those values in scope.
11. **`MemberRow`:** the UI disables role-change/remove entirely for the real owner's own row (`isOwner`), matching the real backend's own server-side guard (`WorkspaceMembersService.requireOwnerIdDoesNotMatch`) — but that UI check is a real courtesy only; the actual enforcement stays server-side.
12. **Workspace Settings page:** fetches the real workspace and its real member list, renders `MemberRow` for each member and `AddMemberForm` below it.
13. **Settings links:** added to the Workspace Detail header (desktop, always visible) and to the mobile `WorkspaceSidebar` drawer (added once the real Settings page existed to link to).

14. **CHALLENGE (written):** a real answer addressing why the httpOnly cookie (not Redux or `localStorage`) holds the real token, why `requireSessionToken()`'s real check runs server-side rather than only hiding UI, and why `currentUserSlice` is a display convenience hydrated FROM an already-authenticated request, never itself an authentication mechanism.

Common mistakes

- Exporting one shared `configureStore(...)` call instead of a factory function — a real concurrency bug under a real server handling many requests at once.
- Reading `useSelector((s) => s.currentUser)` instead of `.user` in `AccountBar`.
- Two sequential `await`s instead of `Promise.all` when resolving reporter/assignee names — works, but needlessly serializes two independent real network calls.
- Routing a real duplicate-slug/duplicate-member error to a thrown exception instead of the real inline form error.
- Debouncing `searchUsersAction`'s own call — a deliberately deferred refinement for a later session, not a bug here.
- Trusting `MemberRow`'s own `isOwner` check as the real security boundary, instead of recognizing it as a UX courtesy backed by a real server-side guard.
- Answering the CHALLENGE with only one of the three required points (the cookie, the layout check, or Redux) instead of explaining how all three genuinely fit together.

Exact testing instructions

BASH

```
# terminal 1
cd api && PORT=3000 npm run start

# terminal 2
cd web && npm run dev

# terminal 3
TOKEN=$(curl -s -X POST http://localhost:3000/auth/login -H "Content-Type: application/json"
\
  -d '{"email":"lina.owner@b4fboard.dev","password":"Password123!"}' | python3 -c \
  "import sys,json;print(json.load(sys.stdin)['accessToken'])")
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1/issues/1 \
  | grep -o "Reporter:[^<]*<[^>]*>[^<]*\|Assignee:[^<]*<[^>]*>[^<]*"
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/settings \
  | grep -o "Lina Haddad\|Omar Rahal\|owner\|Settings"
cd web && npx tsc --noEmit && npm run build
```

Real reporter/assignee names appear on Issue Detail; the real Settings page lists every member, with the real owner's row showing "owner" and no controls; `tsc` / `build` both exit clean.

What you should understand after studying

Why this session kept three genuinely different concerns separate — a server-only httpOnly cookie that holds the real token, a server-side layout check that actually enforces protection, and a client-visible Redux snapshot that only ever displays what the server already decided — and why collapsing any two of them into one would either leak the token to the browser or make the real authorization check something a client could bypass.